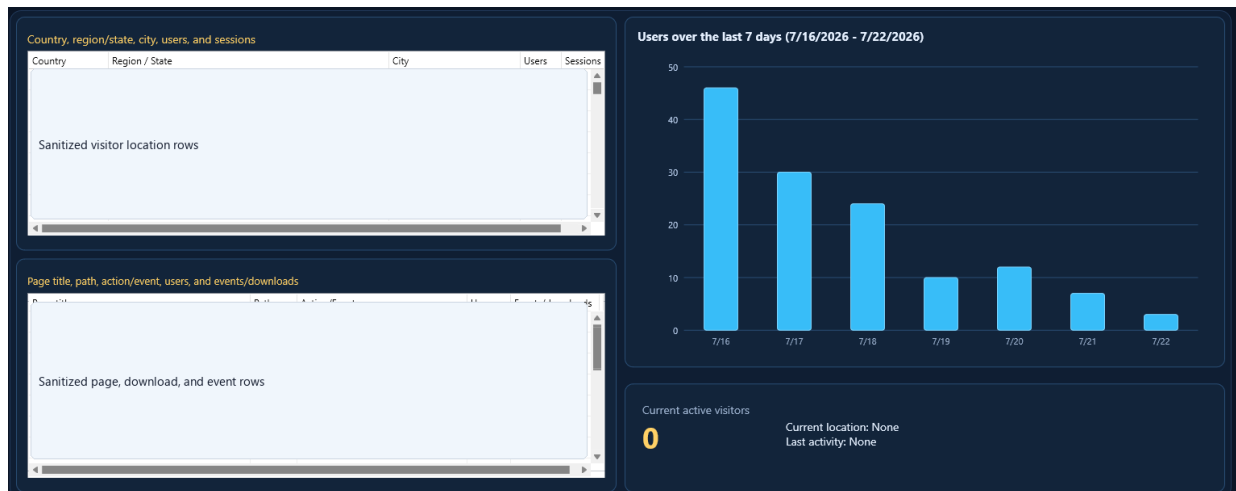


Website Analytics Public How To Guide

Build, configure, authenticate, run, and maintain the GA4 desktop dashboard from scratch

Last changed: July 23, 2026

Written for a public-source repository. Each user supplies their own Google OAuth client and GA4 property IDs.



Current dashboard layout with sanitized example data.

Public-source safety rule

Never publish your local SQLite database, OAuth client secret, encrypted refresh token, credential JSON files, API keys, PEM files, or .env files. Every user should bring their own Google Cloud OAuth desktop client and their own GA4 property access.

Table of contents

Table of contents	2
1. What this application is	4
2. What the current dashboard shows.....	4
3. What the app stores and what it does not store	5
4. Prerequisites.....	6
5. Clone or download the source	6
6. Repository layout.....	6
7. Build the app.....	7
7.1 Build in RAD Studio	7
7.2 Build from command line	7
8. First run and SQLite setup	8
8.1 What should happen on first run	8
9. Find your numeric GA4 property IDs.....	8
10. Add website properties in the app.....	9
10.1 Adding more websites later	10
11. Create your Google Cloud OAuth setup	10
11.1 Create or choose a Google Cloud project.....	10
11.2 Enable Google Analytics Data API	10
11.3 Configure OAuth consent	11
11.4 Create an OAuth 2.0 Desktop client	11
11.5 OAuth values people often confuse	11
12. Configure Google sign-in in Website Analytics.....	12
12.1 Settings fields explained	13
13. Silent login and token refresh	13
14. Run your first dashboard update	14
14.1 What a successful update does.....	14
15. Understand the reports.....	15
15.1 KPI card meanings	17
15.2 Handling no data and '(not set)'	17
16. GA4 data requested by the app.....	17

17. Troubleshooting	18
17.1 A practical troubleshooting order	19
17.2 Error wording to recognize.....	19
18. GitHub setup for contributors	20
18.1 Configure Git identity	20
18.2 Authenticate to GitHub	20
18.3 Normal contribution workflow	20
19. Public release safety checklist	20
20. Google verification for broader public use.....	21
20.1 Practical recommendation for this project	21
20.2 What a privacy explanation should say	21
21. Maintenance rules for developers	22
22. From-zero checklist	22
Appendix A - Source files used to build this guide	22

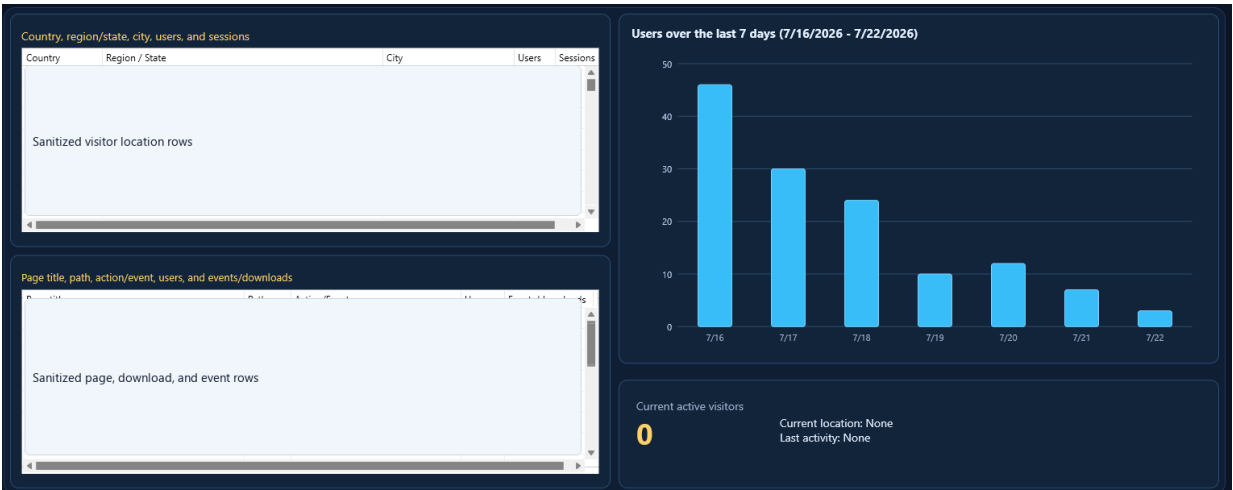
1. What this application is

Website Analytics is a Delphi FireMonkey desktop dashboard for Google Analytics 4. It was built to answer practical website questions without forcing the user through the standard GA4 web console. The program connects to GA4 through Google's Data API, retrieves selected reports, formats the returned rows in memory, and displays them in a cleaner desktop UI.

The application is intentionally focused. It is not a replacement for every GA4 screen. It is a daily operational dashboard: who visited, where they came from, what pages or downloads they used, what the selected date range looks like, and what is happening right now.

Item	Purpose	Public-source instruction
Desktop app	Runs as a Windows Delphi/FM X executable.	Build from source with RAD Studio or an equivalent Delphi toolchain.
GA4 reader	Reads data through Google Analytics Data API.	Each user must authorize their own Google account.
Memory-only reports	Fetches analytics rows are not stored as history.	Close the app and the report rows are gone.
Portable settings	SQLite stores settings beside the project/app.	The generated local database must stay out of Git.

2. What the current dashboard shows



Dashboard overview: locations, pages/downloads/actions, trend graph, realtime panel, and KPI cards.

The current UI is built around a compact dashboard, not a collection of GA4-style pages. The important parts are:

- Hero section: app title, logo, website property selector, date range selector, auto update, connection status, Update, Settings, and Help.

- Location grid: country, region/state, city, users, and sessions for the selected property and date range.
- Pages/downloads/actions panel: page titles, paths, events, users, and event/download totals for the selected range.
- Users graph: proportional bar graph for the selected range, with tooltips for exact values.
- Realtime panel: current active users, current location if GA4 returns it, and last activity time/location.
- KPI cards: users today, users for selected range, views today, downloads for selected range, active now, last location, top page, views last 30 minutes, events per session, and scrolled count.

3. What the app stores and what it does not store

This distinction matters for public-source users and for privacy.

Stored item	Where	Why	Commit to Git?
Website definitions	SQLite settings database	Lets the app know which GA4 properties to query.	No, local database only.
Dashboard defaults	SQLite settings database	Default site, date range, auto-refresh choice, saved grid/card preferences.	No, local database only.
OAuth client ID/secret	SQLite settings database	Convenience for the local user's own Google Cloud OAuth client.	No.
Encrypted refresh token	SQLite settings database	Allows silent reconnect without browser login every launch.	No.
GA4 report rows	Memory only	Displayed, graphed, then discarded when the app closes.	Never stored.
Schema	databases/WebsiteAnalytics.schema.sql	Creates expected settings tables and defaults.	Yes.

Tracked:

databases\WebsiteAnalytics.schema.sql

Ignored local files:

databases*.sqlite3
databases*.sqlite3-journal
databases*.sqlite3-wal
databases*.sqlite3-shm

Refresh token encryption

The app encrypts the saved refresh token with Windows DPAPI. That is suitable for a desktop convenience token, but it does not make the local database a public artifact. Do not commit it.

4. Prerequisites

Requirement	Details
Windows	The current app targets Windows desktop builds.
RAD Studio / Delphi with FireMonkey	Open and edit the .fmx forms in the IDE. Keep forms editable.
RAD Studio toolchain	The project scripts expect the Embarcadero toolchain under C:\Program Files (x86)\Embarcadero\Studio\37.0\bin or a compatible version.
Git	Required for clone, branch, commit, and push workflows.
GitHub account	Required only if you fork, push, or contribute changes.
Google account with GA4 access	The signed-in Google account must have permission to read the GA4 properties you enter.
Google Cloud project	Required to enable the Google Analytics Data API and create an OAuth desktop client.
GA4 properties already collecting data	The app reads GA4; it does not install tracking tags on websites.

5. Clone or download the source

Use Git if you plan to make changes or receive updates.

```
git clone https://github.com/tmartindub/WebsiteAnalytics-GA4-Dashboard.git
cd WebsiteAnalytics-GA4-Dashboard
```

If you download the ZIP from GitHub, extract it to a normal local development folder such as:

```
C:\Projects\WebsiteAnalytics
```

Folder recommendation

Avoid synced folders and restricted paths while building Delphi projects. A simple local folder makes RAD Studio, MSBuild, SQLite, and generated output easier to reason about.

6. Repository layout

Path	Purpose
WebsiteAnalytics.dpr / .dproj	Application entry point and RAD Studio project.
WebsiteAnalytics.MainForm.pas/.fmx	Main editable FMX dashboard, update logic, chart, cards, grids, status, and help/settings launch points.

WebsiteAnalytics.GA4DataModule.pas/.dfm	GA4 report request construction, API calls, and response parsing.
WebsiteAnalytics.AuthenticationDataModule.pas/.dfm	Google OAuth desktop flow, loopback listener, token exchange, refresh-token handling.
WebsiteAnalytics.SettingsDataModule.pas/.dfm	Portable SQLite settings creation, defaults, reads, and writes.
WebsiteAnalytics.SettingsForm.pas/.fmx	OAuth setup, defaults, redirect port, connection/disconnect actions.
WebsiteAnalytics.PropertyManagerForm.pas/.fmx	Website property definitions and GA4 numeric property IDs.
WebsiteAnalytics.Models.pas	Shared data models used between data modules and UI.
databases/WebsiteAnalytics.schema.sql	Tracked database schema for fresh settings database creation.
docs/guides	Editable Word guides.
docs/pdf	Companion PDF guides.
help	Local HTML help and Help Doc Creator project file.
tools	Build, documentation, and maintenance scripts.

7. Build the app

You can build from RAD Studio or from the command line.

7.1 Build in RAD Studio

1. Open WebsiteAnalytics.dproj.
2. Choose the desired platform and configuration, such as Win32 Debug or Win64 Debug.
3. Build the project.
4. Run the executable once to smoke-test form loading and startup behavior.

7.2 Build from command line

The included project build script sets up the Embarcadero environment and builds the project.

```
tools\build_phase1.cmd
```

Expected debug output locations:

```
bin\Win32\Debug\WebsiteAnalytics.exe
bin\Win64\Debug\WebsiteAnalytics.exe
```

Smoke-test after .fmx edits

A compile can succeed while an FMX form stream still fails at runtime. If you change .fmx files, start the EXE and verify it reaches the main window without EReadError exceptions.

8. First run and SQLite setup

On first run, the app creates the portable settings database if it is missing. The repository should ship the schema, not a real user database.

1. Start the app after a successful build.
2. If the databases folder exists but WebsiteAnalytics.sqlite3 is missing, the app creates it.
3. If required settings tables are missing, the app creates or updates them.
4. Open Settings and Properties to enter your own OAuth and GA4 details.

A fresh public-source checkout should not already know your Google credentials or personal website properties. That is intentional.

8.1 What should happen on first run

A clean first run should reach the main dashboard without requiring a database file to already exist. The Settings data module is responsible for creating the database and ensuring the required settings tables exist. If startup fails before the main window appears, the first thing to check is whether a streamed form property, resource, or missing runtime dependency is preventing the executable from loading.

First-run item	Expected result	If it fails
databases folder	Created if missing.	Confirm the application has write access to the project/app folder.
WebsiteAnalytics.sqlite3	Created locally if missing.	Check SQLite/FireDAC availability and folder permissions.
Default settings	Seeded from code/schema defaults.	Check SettingsDataModule initialization.
Generated analytics rows	None stored.	This is correct; report rows are fetched only during updates.

9. Find your numeric GA4 property IDs

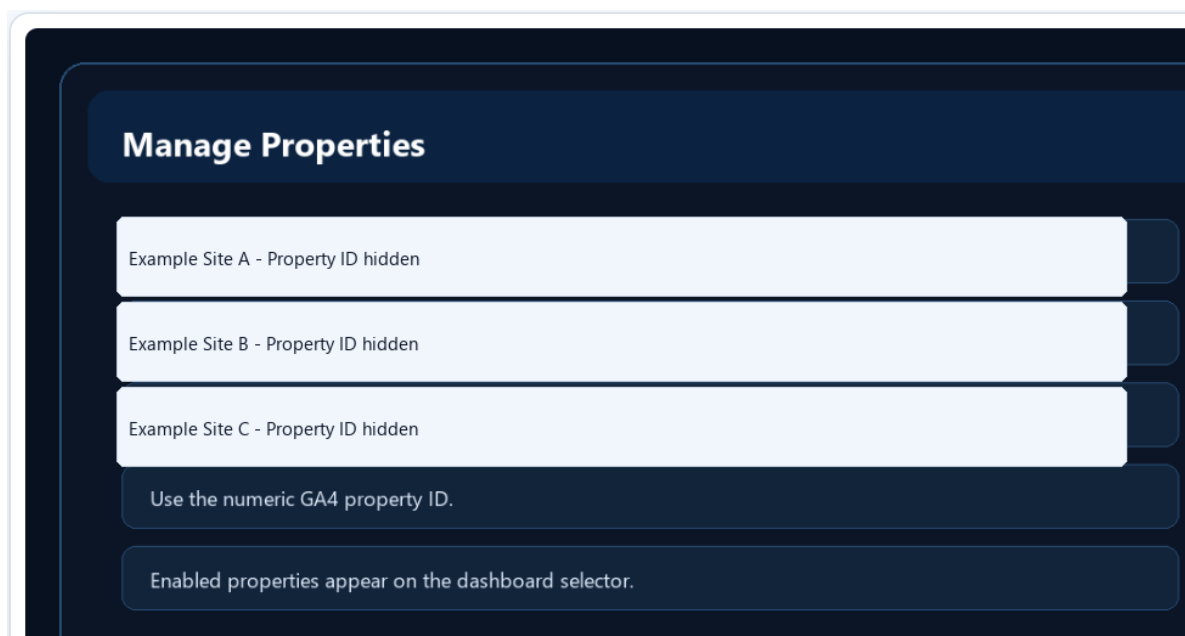
Every website you add needs its numeric GA4 property ID. This is one of the most common setup mistakes, so check it carefully.

Thing	Example format	Where it belongs
GA4 property ID	123456789	Properties screen. This is the number the GA4 Data API uses.

Measurement ID	G-ABCD123456	Your website tracking tag, not this desktop app's property field.
Google Cloud project number	101177308157	Cloud Console only. Do not enter it as a GA4 property ID.
OAuth desktop Client ID	101...apps.googleusercontent.com	Settings screen OAuth Client ID field.
OAuth client secret	Secret string from Google	Settings screen OAuth secret field, hidden by default.
Gmail address	name@example.com	Only the account you sign in with; it is not an app setting.

1. Open Google Analytics.
2. Select the account/property for the website.
3. Open Admin.
4. Open Property details or Property settings.
5. Copy the numeric Property ID.
6. Repeat for each website you want to add.

10. Add website properties in the app



Manage Properties screen with sanitized sample values.

1. Click Settings or Properties, depending on the current UI entry point.
2. Open the property manager.
3. Create or edit a property.

4. Enter a friendly display name.
5. Enter the public website address.
6. Enter the numeric GA4 property ID.
7. Enable the property if it should appear in the dashboard selector.
8. Save the property.

For public screenshots and examples, use placeholder property IDs. Do not publish your real property IDs unless you explicitly choose to.

10.1 Adding more websites later

The application was designed to start with a small set of known properties and then grow. Adding another website should not require code changes. Add a new property definition, enter the numeric GA4 property ID, enable it, and save. The dashboard selector should then include the property after the settings/property list is refreshed.

Field	What to enter	Example for public docs
Display name	Friendly name shown in the dashboard selector.	Example Site
Website URL	The public site address.	https://example.github.io
GA4 property ID	The numeric GA4 property ID.	123456789
Enabled	Whether this property appears in the dashboard.	Checked
Color	Optional UI/report color for the property.	Any non-private color value

11. Create your Google Cloud OAuth setup

A public repository should not ship a shared OAuth client. Each user should create their own Google Cloud project and OAuth desktop client.

11.1 Create or choose a Google Cloud project

1. Open Google Cloud Console.
2. Create a new project or choose an existing project for your own desktop tools.
3. Use a plain name such as Website Analytics Desktop.
4. Confirm you are signed in with the Google account you intend to use.

11.2 Enable Google Analytics Data API

1. Open APIs & Services.
2. Open Library.

3. Search for Google Analytics Data API.
4. Open the API page.
5. Click Enable.

If this API is not enabled, the app may authenticate successfully but fail to update with a 403 API-not-enabled error.

11.3 Configure OAuth consent

1. Open APIs & Services > OAuth consent screen.
2. Choose External unless you are intentionally using an internal Google Workspace-only app.
3. Enter the app name, support email, and developer contact information requested by Google.
4. Use the read-only Analytics scope when requested: <https://www.googleapis.com/auth/analytics.readonly>
5. If the app is in Testing mode, add your Google account as a test user.
6. Save the consent configuration.

Testing versus verified app

For personal use, Testing mode is usually enough if your Google account is listed as a test user. For distribution to other people, do not ask them to use your private testing client. They should create their own client, or the project owner should pursue Google's OAuth app verification process.

11.4 Create an OAuth 2.0 Desktop client

1. Open APIs & Services > Credentials.
2. Click Create credentials.
3. Choose OAuth client ID.
4. Choose Desktop app as the application type.
5. Name it something recognizable.
6. Create it.
7. Copy the Client ID and Client secret.

The Client ID usually ends with `apps.googleusercontent.com`. The Client secret normally begins with a Google-generated secret prefix. They must come from the same OAuth client.

11.5 OAuth values people often confuse

OAuth setup is the part most likely to confuse new users because Google uses several similar-looking identifiers. The desktop app needs exactly two OAuth values in Settings: the OAuth desktop Client ID and the OAuth desktop Client secret. Those are not the same as the Google account email, GA4 property ID, Measurement ID, or Google Cloud project number.

Question	Correct answer
Is my Gmail address the OAuth Client ID?	No. Your Gmail address is only the account you sign in with.
Can I use the Google Cloud project number as the GA4 property ID?	No. The project number belongs to Google Cloud. The GA4 property ID comes from Google Analytics property settings.
Can I use the Measurement ID that starts with G-?	No. That ID goes into website tracking code. The desktop app uses the numeric GA4 property ID.
Can I leave the OAuth secret blank?	No. The desktop client secret must match the OAuth Client ID.
Can the public repository contain one shared secret?	It should not. Each user should create their own OAuth desktop client.

12. Configure Google sign-in in Website Analytics

☒ Retrieve live data when the program opens ☒ Compare with the previous period

GA4 desktop OAuth setup

OAuth desktop client ID

OAuth desktop client ID hidden

OAuth desktop client secret **Loopback port**

OAuth client secret hidden 53682

Redirect URI: `http://127.0.0.1:53682/oauth2redirect`

Connect Google opens your browser. Sign in with the Google account that can read these GA4 properties. The app receives a temporary token for this session only; it does not store your Google password or analytics results.

Connect Google Disconnect Connection status hidden

Visual theme: dark dashboard with high-contrast cards and readable report panels.

Public-safe storage note: settings local, report rows memory-only

Settings and Google sign-in screen with private values redacted.

The default redirect URI is:

```
http://127.0.0.1:53682/oauth2redirect
```

The app uses a loopback listener on the local computer. During sign-in, Google redirects the browser back to this local URI so the desktop app can receive the authorization code.

1. Open Settings.
2. Enter your OAuth desktop Client ID.
3. Use Show only when you need to enter or edit the OAuth Client secret.
4. Enter the OAuth Client secret from the same desktop client.
5. Leave the loopback port at 53682 unless another local program uses that port.

6. Confirm the Redirect URI shown by the app.
7. Click Connect Google.
8. Complete the browser sign-in with the Google account that can read the GA4 properties.
9. If Google shows a testing/unverified warning and this is your own OAuth client, continue intentionally.
10. Return to the app after the browser says you may close the tab.
11. Click Save and Close.

What is saved after sign-in

The app stores settings locally and saves the refresh token encrypted with Windows DPAPI so later startups can reconnect silently. It does not save GA4 report rows.

12.1 Settings fields explained

Setting	Meaning	Public-source guidance
Default website	The property selected when the dashboard opens.	Use a placeholder/default in public docs; users choose their own.
Default date range	Initial reporting window.	Last 7 days is a sensible first setting.
Auto update	Automatically refreshes on the timer while connected.	Useful when the app is left running all day.
OAuth Client ID	Desktop OAuth client identifier.	Each user supplies their own.
OAuth Client secret	Secret paired with the Client ID.	Masked in the UI; do not publish screenshots showing it.
Loopback port	Local HTTP port used for browser redirect.	Default 53682 unless there is a local conflict.
Redirect URI	Local URL Google redirects back to.	Default is <code>http://127.0.0.1:53682/oauth2redirect</code> .

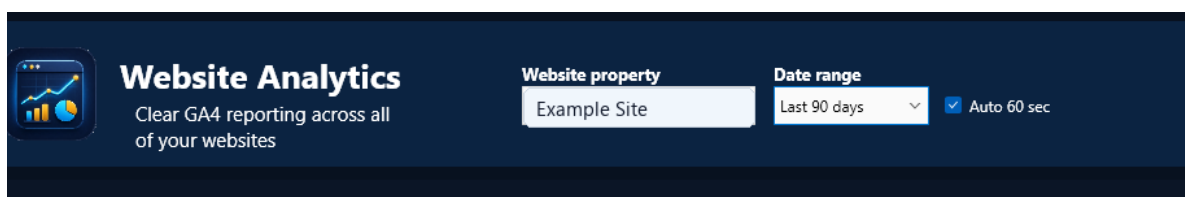
13. Silent login and token refresh

The first authorization needs the browser. After that, the app should normally avoid the repeated Google login/continue screens.

Token or action	What happens
Access token	Short-lived token kept in memory and used for GA4 API calls.
Refresh token	Longer-lived token encrypted with Windows DPAPI and stored in SQLite.
Startup reconnect	The app decrypts the refresh token and asks Google for a fresh access token silently.

Auto update	The 60-second timer can refresh dashboard data without browser prompts while authorization remains valid.
Expired or revoked token	The app falls back to visible Google sign-in.
Disconnect	Clears the in-memory token and saved encrypted refresh token.

14. Run your first dashboard update



Hero section: property selector, date range, auto update, status, Update, Settings, and Help.

1. Choose one website property from the Website property selector.
2. Choose the date range you want to inspect.
3. Click Update.
4. Read the bottom status line if the update fails or returns no data.
5. Hover over graph bars for exact values.
6. Leave Auto 60 sec checked if you want the app to keep refreshing while it remains open.

When All websites is selected, the app queries enabled properties and combines rows in memory. Individual websites are usually clearer when you are diagnosing a specific site's behavior.

14.1 What a successful update does

A successful update is more than a single GA4 call. The app validates authentication, refreshes the access token if needed, resolves the selected property or the enabled property list, runs the requested GA4 reports, merges or normalizes rows where needed, updates the in-memory snapshot, and refreshes the visible grids, cards, graph, realtime panel, and status bar.

Step	What happens
Authentication check	Uses the current access token or silently refreshes it with the encrypted refresh token.
Property resolution	Uses the selected property or builds the enabled-property list for All websites.
Historical reports	Requests summary, trend, location, and content/action rows for the selected date range.
Realtime reports	Requests active-now and recent location/activity information from GA4 realtime reporting.
Normalization	Fills missing dates with zeroes so graphs remain proportional and readable.

Presentation	Updates the grids, cards, graph, realtime text, and status bar.
--------------	---

15. Understand the reports

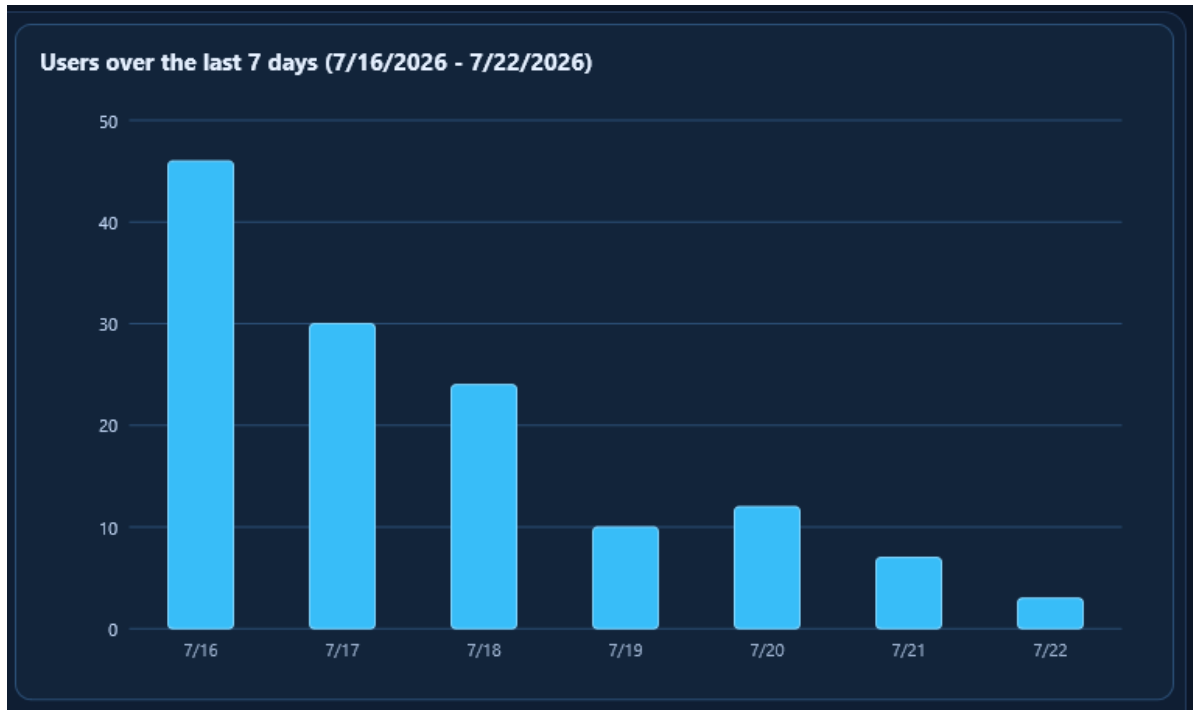
Screen area	Question answered	Typical GA4 source
Location grid	Where did visitors come from?	country, region, city with users and sessions.
Pages/downloads/actions	What content or actions did visitors use?	page title, page path, event name, users, views, event counts, downloads.
Users graph	How did users trend over the selected period?	date plus users, normalized so empty dates show as zero.
Realtime panel	What is happening right now?	runRealtimeReport active users and current location when GA4 returns it.
KPI cards	What are the headline values?	summary, realtime, content, and engagement metrics.

Country, region/state, city, users, and sessions				
Country	Region / State	City	Users	Sessions
Sanitized location rows: country, region/state, city, users, sessions				

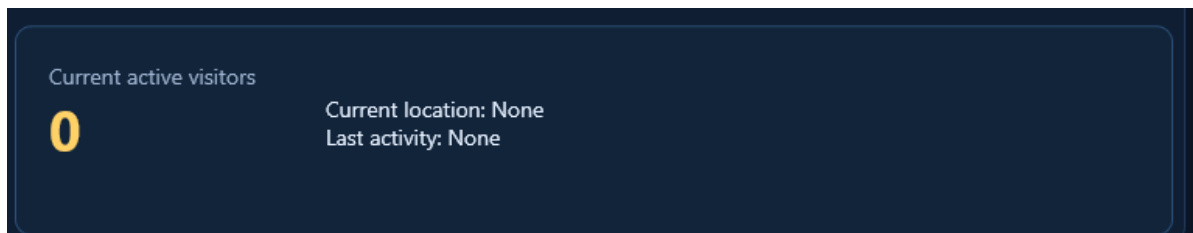
Location grid: country, region/state, city, users, and sessions.

Page title, path, action/event, users, and events/downloads				
Page title	Page path	Action/event	Users	Events/downloads
Sanitized page/download rows: title, path, action/event, users, events				

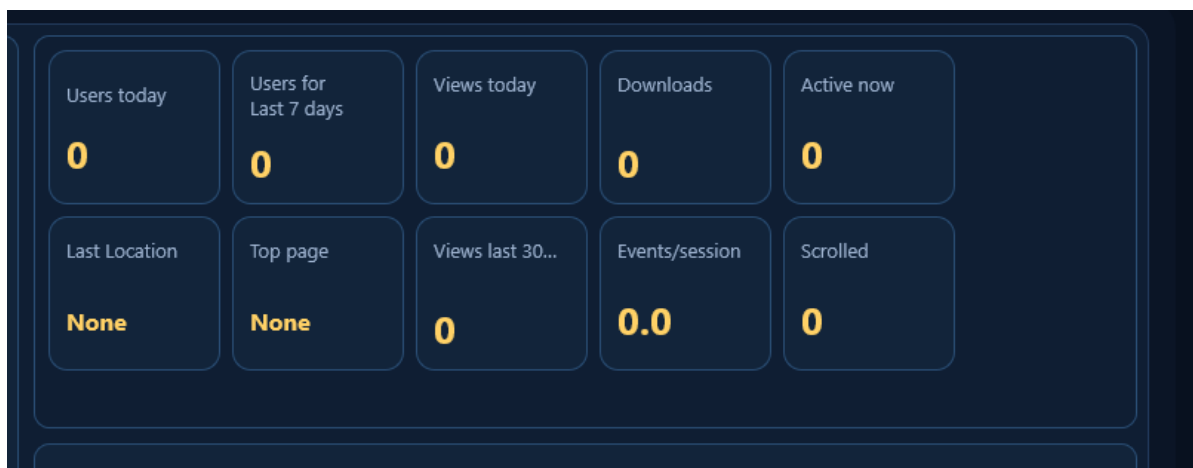
Pages/downloads/actions panel: page title, path, action/event, users, and events/downloads.



Users graph: selected range, whole-number scale, and hover tooltip values.



Realtime panel: current active visitors, current location, and last activity when available.



KPI card area with headline values.

15.1 KPI card meanings

Card	Meaning	Notes
Users today	Users returned by GA4 for today's date.	Standard report data may lag behind realtime.
Users for selected range	Users for the chosen date range.	Changes when date range changes.
Views today	Screen/page views for today.	Useful for quick activity level.
Downloads	Download-related events for the selected range.	Depends on GA4 receiving download events.
Active now	Realtime active users.	Comes from GA4 realtime endpoint.
Last Location	Most recent usable location returned by realtime/location reports.	GA4 may return None or (not set).
Top page	Highest page/action item from current reports.	Home page is shown when path is /.
Views last 30 min	Realtime or near-realtime views in the last 30 minutes.	Can differ from standard report views.
Events/session	Event count divided by sessions where data is available.	Shows activity density.
Scrolled	Scroll events counted for selected range.	Depends on GA4 event collection.

15.2 Handling no data and '(not set)'

GA4 can legitimately return empty rows, zeroes, or '(not set)' values. The app should display those conditions plainly instead of inventing data. A blank result for one date does not necessarily mean the app is broken; it may mean GA4 has no rows for that date/property/report combination, the selected range is too narrow, or Google's processing has not caught up yet.

- Use a wider date range if you expect activity but see no rows.
- Test one property at a time before judging All websites.
- Compare realtime and standard reports with care; they are different GA4 endpoints.
- Treat '(not set)' as a GA4-returned value, not as a local application label.

16. GA4 data requested by the app

The current data module asks GA4 for several focused reports, then translates them into the dashboard snapshot.

Purpose	Dimensions	Metrics	Used for
---------	------------	---------	----------

Summary	None	activeUsers, sessions, screenPageViews, engagementRate	Headline cards.
Trend	date	activeUsers, sessions, screenPageViews, engagementRate, eventCount	Bar graph over selected range.
Geography	country, region, city	activeUsers, sessions, engagementRate	Location grid and last-location style values.
Content/actions	pagePath, pageTitle, eventName	screenPageViews, activeUsers, eventCount, userEngagementDuration	Top pages/downloads/actions panel and cards.
Realtime	country, region, city where available	activeUsers, screenPageViews where available	Active now, current location, views last 30 minutes.
Support reports	source/medium/campaign, device, language	users, sessions, engagement-related metrics	Available for future dashboard expansion.

Realtime can differ from standard reports

GA4 realtime data is not always synchronized with normal report data. The app tries to keep the dashboard internally consistent, but Google's realtime endpoint and standard reporting endpoint do not always update on the same schedule.

17. Troubleshooting

Symptom	Likely cause	What to do
Google login appears every launch	No usable saved refresh token.	Reconnect once from Settings. If it repeats, check whether access was revoked or the database moved to another Windows user/machine.
Access blocked / unverified app	OAuth app is in Testing or not verified.	For private use, add yourself as a test user. For public distribution, create your own OAuth client or pursue verification.
Token exchange failed	Client secret is missing, wrong, or mismatched.	Copy the secret from the same OAuth desktop client as the Client ID.
403 API not enabled	Google Analytics Data API is disabled in the Cloud project.	Enable Google Analytics Data API in APIs & Services > Library.
403 insufficient permissions	Wrong property ID or signed-in Google account cannot read the	Verify the numeric GA4 property ID and GA4 Admin/User access.

	property.	
No rows for a date	No data returned for that property/range.	Try a wider date range or check GA4 web console.
Downloads missing	GA4 is not emitting file_download or custom download events.	Check Enhanced Measurement, outbound/download tracking, and date range.
All websites looks low	One property may fail, no-data may be returned, or rows may be grouped differently.	Test each property individually, then retry All websites and read the status line.
FMX EReadError	Unsupported property was streamed into .fmx.	Remove the invalid property and smoke-test startup.
Shortcut says EXE has no icons	Icon was not embedded into that build target.	Check RAD Studio project icon/resource settings for Win32/Win64 and rebuild the exact EXE you inspect.

17.1 A practical troubleshooting order

1. Check the status bar message first. It usually tells whether the problem is authentication, permission, API access, parsing, or no data.
2. Open Settings and confirm the OAuth Client ID and Client secret belong to the same Desktop app credential.
3. Open Google Cloud Console and confirm Google Analytics Data API is enabled.
4. Open Google Analytics Admin and confirm the signed-in Google account can read the numeric property ID.
5. Try one known-active property instead of All websites.
6. Try Last 90 days instead of Today if you are not sure recent data exists.
7. Open Diagnostics if the app has a request/parser message.
8. If the app will not start, look for .fmx/.dfm EReadError messages and fix unsupported streamed properties.

17.2 Error wording to recognize

Message pattern	Meaning
client_secret is missing	The token exchange reached Google, but the local OAuth secret value is missing or not being sent.
Access blocked / not verified	Google OAuth consent is not verified or the user is not an allowed test user.
Data API has not been used	The Google Analytics Data API is not enabled in the selected Google Cloud project.
User does not have sufficient permissions	The signed-in Google account cannot read the requested GA4 property, or the property ID is wrong.

Value 'rows' not found	The parser expected rows but GA4 returned an empty or different-shaped response. The app should handle this safely.
EReadError reading form	The executable cannot load a streamed Delphi form/resource property.

18. GitHub setup for contributors

A user who only downloads the app does not need GitHub write access. A contributor or maintainer needs Git identity and authentication.

18.1 Configure Git identity

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

18.2 Authenticate to GitHub

Use Git Credential Manager, GitHub CLI, or SSH. GitHub CLI is explicit and easy to verify:

```
gh auth login
# Choose GitHub.com, HTTPS or SSH, then complete the browser/device login.
```

18.3 Normal contribution workflow

```
git clone https://github.com/tmartindub/WebsiteAnalytics-GA4-Dashboard.git
cd WebsiteAnalytics-GA4-Dashboard
git checkout -b my-change

git status
git add <only-the-files-you-intended>
git commit -m "Describe the change"
git push origin my-change
```

Before every push

Run git status. Confirm you are not staging databases/*.sqlite3, OAuth client secrets, encrypted refresh tokens, credential JSON files, .env files, keys, PEM files, or private screenshots.

19. Public release safety checklist

- Keep databases/*.sqlite3 and SQLite sidecar files ignored.
- Keep databases/WebsiteAnalytics.schema.sql tracked.
- Do not commit OAuth client secrets, refresh tokens, generated databases, credential JSON files, keys, PEM files, or .env files.
- Use sanitized screenshots in README, guides, help, issues, and release notes.

- Do not publish private property IDs unless you intentionally decide they are safe to disclose.
- Explain that the OAuth Client ID is not a Gmail address.
- Explain that the GA4 Property ID is not the Measurement ID and not the Google Cloud project number.
- If a secret was ever committed, revoke it in Google Cloud before making the repository public.
- If a secret was committed in history, consider cleaning Git history before public release.
- Expect GitHub secret scanning or safety review to flag OAuth-looking credentials in source, docs, screenshots, or commit history.

20. Google verification for broader public use

For private use, each user creating their own OAuth desktop client is usually the simplest path. For a widely distributed application with a shared OAuth client, Google may require OAuth app verification.

1. Use the least-privileged scope required, normally `analytics.readonly`.
2. Prepare a clear app name, support email, privacy policy, and user-facing explanation of data use.
3. Explain that the app reads GA4 report data and does not store report rows.
4. Provide screenshots or a demonstration video if Google requests them.
5. Submit the OAuth consent app for verification from Google Cloud Console.
6. Do not distribute a testing-mode OAuth client to general public users.

20.1 Practical recommendation for this project

For an open-source desktop utility, the lowest-friction public model is to publish the source code and require each user to create their own Google Cloud OAuth desktop client. That avoids shipping a shared secret and avoids asking strangers to trust a testing-mode OAuth app owned by somebody else. If the project later becomes a packaged product with non-technical users, then a verified OAuth app and a polished privacy policy become much more important.

20.2 What a privacy explanation should say

- The app requests read-only GA4 access.
- The app uses GA4 report data to display local dashboard panels.
- The app does not upload analytics data to a third-party service.
- The app does not store fetched analytics report rows as history.
- The app stores local settings and an encrypted refresh token so it can reconnect.
- Disconnect clears the saved token.

21. Maintenance rules for developers

- Keep FMX forms editable in RAD Studio. Prefer Object Inspector settings for layout and visual values unless runtime behavior is truly necessary.
- Do not add helper code unless it has a clear purpose and the project owner approves it.
- Back up before project changes when working under the project directives.
- After approved changes are complete and validated, commit and push the intended files.
- Build and smoke-test after form changes, resource/icon changes, OAuth changes, GA4 parser changes, and documentation generator changes.
- Regenerate the user guide, engineering guide, help, and this How To guide when the UI or setup flow changes.

22. From-zero checklist

- Clone or download the public repository.
- Open or build WebsiteAnalytics.dproj.
- Run the app once so the local settings database is created.
- Find the numeric GA4 property IDs for your websites.
- Add your website properties in the app.
- Create or choose a Google Cloud project.
- Enable Google Analytics Data API.
- Configure OAuth consent and add yourself as a test user if needed.
- Create an OAuth 2.0 Desktop client.
- Enter the OAuth Client ID and Client secret in Settings.
- Confirm the redirect URI is `http://127.0.0.1:53682/oauth2redirect` unless you changed the port.
- Connect Google once through the browser.
- Save Settings.
- Choose a property and date range.
- Click Update.
- Leave Auto 60 sec enabled if you want a live dashboard.
- Keep your generated database and credentials out of Git.

Appendix A - Source files used to build this guide

This guide was built from the current project source, forms, schema, README, help topics, and documentation generators. The most relevant files for future maintenance are listed below.

- README.md
- WebsiteAnalytics.dpr and WebsiteAnalytics.dproj
- WebsiteAnalytics.MainForm.pas/.fmx

- WebsiteAnalytics.SettingsForm.pas/.fmx
- WebsiteAnalytics.PropertyManagerForm.pas/.fmx
- WebsiteAnalytics.AuthenticationDataModule.pas/.dfm
- WebsiteAnalytics.GA4DataModule.pas/.dfm
- WebsiteAnalytics.SettingsDataModule.pas/.dfm
- WebsiteAnalytics.AnalyticsMemoryDataModule.pas/.dfm
- WebsiteAnalytics.Models.pas
- databases/WebsiteAnalytics.schema.sql
- help/WebsiteAnalytics Help.hdc.json
- help/topics/*.html and help/assets/*.png
- tools/build_phase1.cmd
- tools/build_public_how_to_guide.py
- tools/refresh_websiteanalytics_docs.py